Python Practical Examination - Beginner-Friendly Solutions

Sanjivani University, Kopargaon | Integrated M.Tech (IMtech) Simplified & Easy-to-Learn Python Code

> **Beginner-Friendly Version:** All 20 programs are rewritten with standard loops, direct variable names, simple `if-else` structures, and plain logic to ensure quick comprehension during exams.

Q1. Text Analytics Engine (No re, No collections)

Cleans text using basic string methods, counts word frequencies, finds the longest word, calculates average length, and detects palindromes.

```
# Simple Text Analytics without re or collections
def analyze_text(text):
    if text.strip() == "":
        print("Error: Input text is empty!")
        return

    # Step 1: Clean punctuation and convert to lowercase
    punctuations = "!@#$%^&*()-_=+[]{}|;:'\",.<>/?`~"
    clean_text = ""
    for char in text.lower():
        if char in punctuations:
            clean_text += " "
        else:
            clean_text += char

    # Step 2: Tokenize using split()
    words = clean_text.split()
    if len(words) == 0:
        print("No valid words found!")
        return

    # Step 3: Word Frequency
    freq = {}
    for w in words:
        if w in freq:
            freq[w] += 1
        else:
            freq[w] = 1

    # Step 4: Top 5 words
    sorted_words = sorted(freq.items(), key=lambda x: x[1], reverse=True)
    top_5 = sorted_words[:5]

    # Step 5: Longest word, average length, palindromes
    longest = words[0]
    total_length = 0
    palindromes = []

    for w in words:
        total_length += len(w)
        if len(w) > len(longest):
            longest = w
        if len(w) > 1 and w == w[::-1]:
            if w not in palindromes:
                palindromes.append(w)

    avg_length = total_length / len(words)

    # Step 6: Print report
    print("\n----- TEXT REPORT -----")
    print("Total Words:", len(words))
    print("Unique Words:", len(freq))
    print("Longest Word:", longest)
    print("Average Word Length:", round(avg_length, 2))
    print("Palindromic Words:", palindromes)
    print("Top 5 Words:")
    for word, count in top_5:
        print(" ", word, ":", count)

# Test the function
sample = "Madam Arora teaches radar systems. Wow, radar is very good and madam is nice."
analyze_text(sample)
analyze_text("   ")  # Test empty case
```

Q2. Matrix Operations Using Nested Lists (No NumPy)

Matrix addition, subtraction, multiplication, and transpose using nested `for` loops with simple dimension checks.

```
def print_matrix(mat, title="Matrix"):
    print(f"\n--- {title} ---")
    for row in mat:
        for val in row:
            print(val, end="\t")
        print()

def add_matrices(A, B):
    r1, c1 = len(A), len(A[0])
    r2, c2 = len(B), len(B[0])
    if r1 != r2 or c1 != c2:
        print("Error: Addition not possible! Dimensions do not match.")
        return None
    res = []
    for i in range(r1):
        row = []
        for j in range(c1):
            row.append(A[i][j] + B[i][j])
        res.append(row)
    return res

def multiply_matrices(A, B):
    r1, c1 = len(A), len(A[0])
    r2, c2 = len(B), len(B[0])
    if c1 != r2:
        print("Error: Multiplication not possible! Columns of A != Rows of B.")
        return None
    res = []
    for i in range(r1):
        row = []
        for j in range(c2):
            total = 0
            for k in range(c1):
                total += A[i][k] * B[k][j]
            row.append(total)
        res.append(row)
    return res

def transpose_matrix(A):
    rows, cols = len(A), len(A[0])
    res = []
    for j in range(cols):
        row = []
        for i in range(rows):
            row.append(A[i][j])
        res.append(row)
    return res

# Test cases
A = [[1, 2, 3], [4, 5, 6]]
B = [[7, 8, 9], [1, 2, 3]]
C = [[1, 2], [3, 4], [5, 6]]

print_matrix(add_matrices(A, B), "A + B")
print_matrix(multiply_matrices(A, C), "A * C")
print_matrix(transpose_matrix(A), "Transpose of A")
```

Q3. Student Record Management

Manages student data via nested dictionaries, handles adding/deleting, marks grading (<35 fails), topper search, and department averages.

```
students = {}

def add_student(roll, name, dept, marks):
    if roll in students:
        print("Error: Roll number already exists!")
        return
    students[roll] = {"name": name, "dept": dept, "marks": marks}
    print(f"Student {name} added successfully.")

def calculate_result(marks):
    total = sum(marks)
    pct = total / 3.0
    if any(m < 35 for m in marks):
        status = "Fail"
        grade = "F"
    else:
        status = "Pass"
        if pct >= 75:
            grade = "A"
        elif pct >= 60:
            grade = "B"
        else:
            grade = "C"
    return total, round(pct, 2), grade, status

def display_rank_list():
    ranks = []
    for roll, data in students.items():
        total, pct, grade, status = calculate_result(data["marks"])
        ranks.append((roll, data["name"], data["dept"], pct, grade, status))
    # Sort by percentage descending
    ranks.sort(key=lambda x: x[3], reverse=True)
    print("\n--- STUDENT RANK LIST ---")
    for r in ranks:
        print(f"Roll: {r[0]} | Name: {r[1]} | Dept: {r[2]} | Pct: {r[3]}% | Grade: {r[4]} |
Status: {r[5]}")

def department_averages():
    dept_totals = {}
    dept_counts = {}
    for roll, data in students.items():
        d = data["dept"]
        _, pct, _, _ = calculate_result(data["marks"])
        dept_totals[d] = dept_totals.get(d, 0) + pct
        dept_counts[d] = dept_counts.get(d, 0) + 1
    print("\n--- DEPT AVERAGES ---")
    for d in dept_totals:
        print(f"{d}: {round(dept_totals[d] / dept_counts[d], 2)}%")

# Add sample records
add_student(101, "Rahul", "CSE", [80, 85, 90])
add_student(102, "Sneha", "IT", [30, 75, 80])  # One subject < 35
add_student(103, "Amit", "CSE", [70, 65, 75])
display_rank_list()
department_averages()
```

Q4. Recursion Laboratory

Basic recursive solutions for Tower of Hanoi, Binary Search on sorted lists, and string permutations.

```
# (a) Tower of Hanoi
move_count = 0
def hanoi(n, source, target, helper):
    global move_count
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        move_count += 1
        return
    hanoi(n - 1, source, helper, target)
    print(f"Move disk {n} from {source} to {target}")
    move_count += 1
    hanoi(n - 1, helper, target, source)

# (b) Recursive Binary Search
def binary_search(arr, low, high, key):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == key:
        return mid
    elif arr[mid] > key:
        return binary_search(arr, low, mid - 1, key)
    else:
        return binary_search(arr, mid + 1, high, key)

# (c) String Permutations
def get_permutations(s):
    if len(s) <= 1:
        return [s]
    result = []
    for i in range(len(s)):
        char = s[i]
        remaining = s[:i] + s[i+1:]
        for p in get_permutations(remaining):
            if char + p not in result:
                result.append(char + p)
    return result

# Test
print("Tower of Hanoi (3 disks):")
hanoi(3, 'A', 'C', 'B')
print("Total Moves:", move_count)

nums = [10, 20, 30, 40, 50, 60]
pos = binary_search(nums, 0, len(nums) - 1, 40)
print(f"Binary search for 40 found at index: {pos}")
print("Permutations of 'CAT':", get_permutations("CAT"))
```

Q5. Course Recommendation Using Sets

Uses basic set operations (&, |, -) to calculate Jaccard similarity and recommend courses.

```
user_courses = {
    "Amit": {"Python", "DSA", "ML"},
    "Pooja": {"Python", "Web", "DB"},
    "Rohan": {"Python", "DSA", "Cloud"},
    "Sara": {"ML", "AI", "Python"},
    "Kunal": {"Java", "DSA", "Web"},
    "Neha": {"Python", "DSA", "Web"}
}

def jaccard_similarity(user1, user2):
    s1 = user_courses[user1]
    s2 = user_courses[user2]
    intersection = len(s1 & s2)
    union = len(s1 | s2)
    return intersection / union if union > 0 else 0

def recommend_courses(target_user):
    if target_user not in user_courses:
        print("User not found!")
        return
    best_sim = -1
    most_similar_user = None
    for other in user_courses:
        if other != target_user:
            sim = jaccard_similarity(target_user, other)
            if sim > best_sim:
                best_sim = sim
                most_similar_user = other

    recs = user_courses[most_similar_user] - user_courses[target_user]
    print(f"Recommendations for {target_user} (based on {most_similar_user}):", list(recs)[:3])

def most_popular_course():
    counts = {}
    for c_set in user_courses.values():
        for course in c_set:
            counts[course] = counts.get(course, 0) + 1
    popular = max(counts.items(), key=lambda x: x[1])
    print(f"Most popular course: {popular[0]} ({popular[1]} users)")

recommend_courses("Amit")
most_popular_course()
```

Q6. Sales Data File Processing

Safe file operations using `open()` with `try...except`, row validations, regional total calculation, and output saving.

```
def process_sales(in_file="sales.txt", out_file="summary.txt"):
    try:
        f = open(in_file, "r")
        lines = f.readlines()
        f.close()
    except FileNotFoundError:
        print("Error: Input file does not exist!")
        return

    regional_revenue = {}
    valid_rows = []

    for line in lines:
        parts = line.strip().split(",")
        if len(parts) != 5:
            continue
        sid, region, product, price_str, qty_str = parts
        try:
            price = float(price_str.strip())
            qty = int(qty_str.strip())
            if price <= 0 or qty <= 0:
                continue
            rev = price * qty
            valid_rows.append(f"{sid.strip()}, {region.strip()}, {product.strip()}, {rev}")
            r = region.strip()
            regional_revenue[r] = regional_revenue.get(r, 0) + rev
        except ValueError:
            continue

    if not regional_revenue:
        print("No valid records found.")
        return

    best_region = max(regional_revenue.items(), key=lambda x: x[1])

    out = open(out_file, "w")
    out.write("--- VALID SALES RECORDS ---\n")
    for r in valid_rows:
        out.write(r + "\n")
    out.write("\n--- REGIONAL REVENUE ---\n")
    for reg, tot in regional_revenue.items():
        out.write(f"{reg}: {tot}\n")
    out.write(f"\nHighest Revenue Region: {best_region[0]} ({best_region[1]})\n")
    out.close()
    print("Sales processed and saved to", out_file)

# Write sample data and run
with open("sales.txt", "w") as f:
    f.write("1, North, Pen, 10, 5\n2, South, Book, -20, 2\n3, North, Eraser, 5, 10\n")
process_sales()
```

Q7. Bank Transaction Processor

Reads transactions, tracks balances with simple dictionary, catches custom exception on low balance, and reports rejected rows.

```
class InsufficientBalanceError(Exception):
    pass

def process_bank(filename="transactions.txt"):
    try:
        f = open(filename, "r")
        lines = f.readlines()
        f.close()
    except FileNotFoundError:
        print("Error: File not found!")
        return

    balances = {}
    rejected = 0

    for line in lines:
        parts = line.strip().split(",")
        if len(parts) != 3:
            rejected += 1
            continue
        acc, tx_type, amt_str = parts[0].strip(), parts[1].strip().lower(), parts[2].strip()
        try:
            amt = float(amt_str)
            if amt <= 0:
                rejected += 1
                continue
            if acc not in balances:
                balances[acc] = 0.0

            if tx_type == "deposit":
                balances[acc] += amt
            elif tx_type == "withdraw":
                if balances[acc] < amt:
                    raise InsufficientBalanceError(f"Acc {acc} has low balance")
                balances[acc] -= amt
            else:
                rejected += 1
        except (InsufficientBalanceError, ValueError) as e:
            rejected += 1
            print("Rejected Transaction:", e)

    print("\n--- FINAL BALANCES ---")
    for acc, bal in balances.items():
        print(f"Account {acc}: Rs. {bal}")
    print("Total Rejected Transactions:", rejected)

with open("transactions.txt", "w") as f:
    f.write("A1, deposit, 500\nA1, withdraw, 200\nA2, withdraw, 100\n")
process_bank()
```

Q8. Library Management System

Classes Book, `Member`, and `Library` enforcing max 3 books limit, duplicate checkout prevention, and overdue fine calculation.

```
class Book:
    def __init__(self, book_id, title):
        self.book_id = book_id
        self.title = title
        self.is_issued = False

class Member:
    def __init__(self, member_id, name):
        self.member_id = member_id
        self.name = name
        self.borrowed = {}  # book_id: issue_day
        self.fine = 0

class Library:
    def __init__(self):
        self.books = {}
        self.members = {}

    def issue_book(self, member_id, book_id, current_day):
        m = self.members[member_id]
        b = self.books[book_id]
        if b.is_issued:
            print(f"Error: {b.title} is already issued!")
            return
        if len(m.borrowed) >= 3:
            print("Error: Max 3 books limit reached!")
            return
        b.is_issued = True
        m.borrowed[book_id] = current_day
        print(f"Issued '{b.title}' to {m.name}")

    def return_book(self, member_id, book_id, return_day):
        m = self.members[member_id]
        b = self.books[book_id]
        issue_day = m.borrowed.pop(book_id)
        b.is_issued = False
        days_kept = return_day - issue_day
        if days_kept > 14:
            late_fee = (days_kept - 14) * 2
            m.fine += late_fee
            print(f"Returned late. Fine added: Rs. {late_fee}")
        else:
            print("Returned on time. No fine.")

lib = Library()
lib.books["B1"] = Book("B1", "Learn Python")
lib.members["M1"] = Member("M1", "Aakash")
lib.issue_book("M1", "B1", 1)
lib.return_book("M1", "B1", 20)  # Kept 19 days (5 days late -> Rs. 10 fine)
```

Q9. Payroll System Using Polymorphism

Base class `Employee` with overridden `calculate_pay()` in `FullTime`, `PartTime`, and `Intern` subclasses.

```
class Employee:
    def __init__(self, name):
        self.name = name
    def calculate_pay(self):
        return 0

class FullTime(Employee):
    def __init__(self, name, monthly_salary):
        super().__init__(name)
        self.salary = monthly_salary
    def calculate_pay(self):
        return self.salary

class PartTime(Employee):
    def __init__(self, name, hours, hourly_rate):
        super().__init__(name)
        self.hours = hours
        self.rate = hourly_rate
    def calculate_pay(self):
        return self.hours * self.rate

class Intern(Employee):
    def __init__(self, name, stipend):
        super().__init__(name)
        self.stipend = stipend
    def calculate_pay(self):
        return self.stipend

# Polymorphism demonstration
employees = [
    FullTime("Suresh", 50000),
    PartTime("Priyanka", 40, 500),
    Intern("Ramesh", 15000)
]

total = 0
for emp in employees:
    pay = emp.calculate_pay()  # Same method name, different behavior
    total += pay
    print(f"Employee: {emp.name} | Pay: Rs. {pay}")

print("Average Salary: Rs.", total / len(employees))
```

Q10. Secure Wallet System

Encapsulation using private variables (`__pin`, `__balance`), 3 incorrect attempts lockout, and transaction log.

```
class Wallet:
    def __init__(self, pin, balance=0.0):
        self.__pin = str(pin)
        self.__balance = float(balance)
        self.__attempts = 0
        self.__locked = False
        self.__history = []

    def __check_pin(self, pin):
        if self.__locked:
            print("Wallet is locked due to too many failed attempts!")
            return False
        if str(pin) == self.__pin:
            self.__attempts = 0
            return True
        else:
            self.__attempts += 1
            print(f"Wrong PIN! Attempts left: {3 - self.__attempts}")
            if self.__attempts >= 3:
                self.__locked = True
                print("Wallet is now LOCKED!")
            return False

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            self.__history.append(f"Deposited: Rs. {amount}")
            print(f"Added Rs. {amount}")

    def withdraw(self, pin, amount):
        if self.__check_pin(pin):
            if amount > self.__balance:
                print("Error: Insufficient balance!")
            else:
                self.__balance -= amount
                self.__history.append(f"Withdrawn: Rs. {amount}")
                print(f"Withdrawn Rs. {amount}")

    def show_history(self, pin):
        if self.__check_pin(pin):
            print("\n--- HISTORY ---")
            for h in self.__history:
                print(h)
            print("Current Balance: Rs.", self.__balance)

w = Wallet("1234", 1000)
w.deposit(500)
w.withdraw("1234", 200)
w.show_history("1234")
```

Q11. Multiple Inheritance and MRO

Demonstrates multiple inheritance with `super()`: `TeachingAssistant(Student, Employee)` and displays `__mro__`.

```
class Person:
    def __init__(self, name, age, **kwargs):
        super().__init__(**kwargs)
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, roll_no, **kwargs):
        super().__init__(**kwargs)
        self.roll_no = roll_no

class Employee(Person):
    def __init__(self, emp_id, base_salary, **kwargs):
        super().__init__(**kwargs)
        self.emp_id = emp_id
        self.base_salary = base_salary

class TeachingAssistant(Student, Employee):
    def __init__(self, name, age, roll_no, emp_id, base_salary, teaching_hours):
        super().__init__(name=name, age=age, roll_no=roll_no, emp_id=emp_id,
base_salary=base_salary)
        self.teaching_hours = teaching_hours

    def calculate_stipend(self):
        # Base salary + 300 per teaching hour
        return self.base_salary + (self.teaching_hours * 300)

    def display(self):
        print(f"TA: {self.name} | Roll: {self.roll_no} | ID: {self.emp_id} | Stipend: Rs.
{self.calculate_stipend()}")

ta1 = TeachingAssistant("Rohit", 23, "R101", "E201", 10000, 15)
ta2 = TeachingAssistant("Kavita", 24, "R102", "E202", 12000, 20)
ta1.display()
ta2.display()

print("\nMethod Resolution Order (MRO):")
for cls in TeachingAssistant.__mro__:
    print(cls.__name__)
```

Q12. Polynomial Class with Operator Overloading

Overloads +, -, *, ( ) call, and generates derivative using list of coefficients.

```
class Polynomial:
    def __init__(self, coeffs):
        # Coefficients stored in index order: coeffs[0] + coeffs[1]*x + coeffs[2]*x^2
        self.coeffs = list(coeffs)

    def __str__(self):
        terms = []
        for power, c in enumerate(self.coeffs):
            if c != 0:
                if power == 0:
                    terms.append(f"{c}")
                elif power == 1:
                    terms.append(f"{c}x")
                else:
                    terms.append(f"{c}x^{power}")
        return " + ".join(terms) if terms else "0"

    def __add__(self, other):
        max_len = max(len(self.coeffs), len(other.coeffs))
        res = []
        for i in range(max_len):
            c1 = self.coeffs[i] if i < len(self.coeffs) else 0
            c2 = other.coeffs[i] if i < len(other.coeffs) else 0
            res.append(c1 + c2)
        return Polynomial(res)

    def __call__(self, x):
        # Evaluate for x
        val = 0
        for power, c in enumerate(self.coeffs):
            val += c * (x ** power)
        return val

    def derivative(self):
        # d/dx (c * x^p) = p * c * x^(p-1)
        res = []
        for power in range(1, len(self.coeffs)):
            res.append(power * self.coeffs[power])
        return Polynomial(res if res else [0])

p1 = Polynomial([3, 2, 1])  # 3 + 2x + 1x^2
p2 = Polynomial([1, 4])     # 1 + 4x
print("P1:", p1)
print("P2:", p2)
print("P1 + P2:", p1 + p2)
print("P1(2) =", p1(2))
print("Derivative of P1:", p1.derivative())
```

Q13. Inventory Manager

Classes `Product` and `Inventory` managing stock, rejecting excessive selling, and checking low inventory.

```
class Product:
    def __init__(self, code, name, price, qty):
        self.code = code
        self.name = name
        self.price = price
        self.qty = qty

class Inventory:
    def __init__(self):
        self.products = {}

    def add_product(self, product):
        self.products[product.code] = product

    def restock(self, code, qty):
        if code in self.products:
            self.products[code].qty += qty
            print(f"Restocked {qty} items for {code}")

    def sell(self, code, qty):
        if code not in self.products:
            print("Product not found!")
            return
        p = self.products[code]
        if qty > p.qty:
            print(f"Error: Not enough stock! (Available: {p.qty})")
        else:
            p.qty -= qty
            print(f"Sold {qty} units of {p.name}")

    def total_value(self):
        return sum(p.price * p.qty for p in self.products.values())

    def show_low_stock(self, limit=5):
        print(f"\nProducts with quantity below {limit}:")
        for p in self.products.values():
            if p.qty < limit:
                print(f" - {p.name} (Qty: {p.qty})")

inv = Inventory()
inv.add_product(Product("P1", "Keyboard", 500, 10))
inv.add_product(Product("P2", "Mouse", 250, 3))
inv.sell("P1", 2)
inv.sell("P2", 5)  # Triggers not enough stock
inv.show_low_stock(5)
print("Total Inventory Value: Rs.", inv.total_value())
```

Q14. Hospital Appointment System

Classes `Doctor`, `Patient`, and `Hospital` preventing double slot booking and allowing cancellation.

```
class Doctor:
    def __init__(self, doc_id, name):
        self.doc_id = doc_id
        self.name = name

class Patient:
    def __init__(self, pat_id, name):
        self.pat_id = pat_id
        self.name = name

class HospitalSystem:
    def __init__(self):
        self.appointments = {}  # slot_key: patient_name

    def book(self, doctor, patient, time_slot):
        slot_key = (doctor.doc_id, time_slot)
        if slot_key in self.appointments:
            print(f"Error: Slot {time_slot} is ALREADY BOOKED for Dr. {doctor.name}!")
        else:
            self.appointments[slot_key] = patient.name
            print(f"Booked appointment for {patient.name} with Dr. {doctor.name} at
{time_slot}")

    def cancel(self, doctor, time_slot):
        slot_key = (doctor.doc_id, time_slot)
        if slot_key in self.appointments:
            removed = self.appointments.pop(slot_key)
            print(f"Cancelled appointment of {removed} at {time_slot}")
        else:
            print("No appointment found to cancel.")

doc = Doctor("D1", "Verma")
p1 = Patient("P1", "Aditya")
p2 = Patient("P2", "Sunita")

hosp = HospitalSystem()
hosp.book(doc, p1, "10:00 AM")
hosp.book(doc, p2, "10:00 AM")  # Attempt duplicate slot
hosp.cancel(doc, "10:00 AM")
hosp.book(doc, p2, "10:00 AM")  # Successful after cancellation
```

Q15. Signal Processing with NumPy

Noisy sine wave generation, moving-average smoothing with `np.convolve`, outlier detection via Z-score, and plotting.

```
import numpy as np
import matplotlib.pyplot as plt

# (a) Generate 300 samples with noise
np.random.seed(42)
t = np.linspace(0, 10, 300)
signal = np.sin(t) + np.random.normal(0, 0.3, 300)

# (b) Moving-average filter (window = 5)
window = np.ones(5) / 5
smoothed = np.convolve(signal, window, mode='same')

# (c) Identify outliers where |z-score| > 2
mean = np.mean(smoothed)
std = np.std(smoothed)
z_scores = np.abs((smoothed - mean) / std)
outliers = z_scores > 2

# (d) Replace outliers with average of smoothed
cleaned = smoothed.copy()
cleaned[outliers] = mean

# (e) Plot results using object-oriented style
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(t, signal, label="Original Noisy Signal", color="gray", alpha=0.6)
ax.plot(t, cleaned, label="Processed Signal", color="blue")
ax.set_title("NumPy Signal Processing")
ax.set_xlabel("Time")
ax.set_ylabel("Amplitude")
ax.legend()
plt.tight_layout()
plt.savefig("signal_plot.png")
print("Signal plot saved as signal_plot.png")
```

Q16. Business Data Analysis with Pandas

Missing value fill with median, calculated column `Revenue = Units * Price`, regional `groupby()`, and CSV export.

```
import pandas as pd
import numpy as np

# Create 20 sample records
data = {
    "Date": pd.date_range("2026-01-01", periods=20),
    "Region": ["North", "South", "East", "West"] * 5,
    "Product": ["Pen", "Book", "Pen", "Bag"] * 5,
    "Units": [10, 20, np.nan, 15, 12, 18, 25, 30, 8, 14, 22, 16, 19, 11, 24, 28, 13, 17, 21,
15],
    "Price": [10, 50, 10, 100] * 5
}
df = pd.DataFrame(data)

# (a) Handle missing values
df["Units"] = df["Units"].fillna(df["Units"].median())

# (b) Calculate Revenue
df["Revenue"] = df["Units"] * df["Price"]

# (c) Groupby Region and Product
print("--- Total Revenue by Region ---")
print(df.groupby("Region")["Revenue"].sum())

print("\n--- Total Revenue by Product ---")
prod_rev = df.groupby("Product")["Revenue"].sum()
print(prod_rev)

# (d) Highest revenue product
print("\nTop Product:", prod_rev.idxmax())

# (e) Export to CSV
df.to_csv("final_sales.csv", index=False)
print("Saved DataFrame to final_sales.csv")
```

Q17. Multi-Panel Analytical Dashboard (Matplotlib)

Creates a 2x2 multi-panel figure using `fig, axes = plt.subplots(2, 2)` with line plot, histogram, scatter, and bar chart.

```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(2, 2, figsize=(8, 6))

# Panel 1: Line plot
x = np.arange(1, 11)
y = x * 2
axes[0, 0].plot(x, y, color="blue", marker="o")
axes[0, 0].set_title("Panel 1: Growth Trend")

# Panel 2: Histogram with Mean
data = np.random.normal(50, 5, 100)
axes[0, 1].hist(data, bins=10, color="orange", edgecolor="black")
axes[0, 1].axvline(data.mean(), color="red", linestyle="--", label="Mean")
axes[0, 1].set_title("Panel 2: Distribution")
axes[0, 1].legend()

# Panel 3: Scatter Plot with polyfit line
xs = np.array([1, 2, 3, 4, 5, 6, 7])
ys = np.array([2, 5, 7, 8, 11, 13, 15])
m, c = np.polyfit(xs, ys, 1)
axes[1, 0].scatter(xs, ys, color="green")
axes[1, 0].plot(xs, m * xs + c, color="black", linestyle="--")
axes[1, 0].set_title("Panel 3: Regression")

# Panel 4: Grouped Bar Chart
labels = ["A", "B", "C"]
g1 = [10, 20, 30]
g2 = [15, 25, 20]
ind = np.arange(len(labels))
axes[1, 1].bar(ind - 0.2, g1, width=0.4, label="Class 1")
axes[1, 1].bar(ind + 0.2, g2, width=0.4, label="Class 2")
axes[1, 1].set_xticks(ind)
axes[1, 1].set_xticklabels(labels)
axes[1, 1].set_title("Panel 4: Comparison")
axes[1, 1].legend()

plt.tight_layout()
plt.savefig("dashboard.png")
print("Saved dashboard to dashboard.png")
```

Q18. Plot Class Framework

Base class `BasePlot` with `LinePlot` and `BarPlot`, displayed via `Dashboard` using subplots.

```
import matplotlib.pyplot as plt

class BasePlot:
    def __init__(self, x, y, title):
        self.x = x
        self.y = y
        self.title = title

    def render(self, ax):
        pass

class LinePlot(BasePlot):
    def render(self, ax):
        ax.plot(self.x, self.y, color="blue", marker="o")
        ax.set_title(self.title)

class BarPlot(BasePlot):
    def render(self, ax):
        ax.bar(self.x, self.y, color="green")
        ax.set_title(self.title)

class Dashboard:
    def __init__(self, plots):
        self.plots = plots

    def show(self):
        fig, axes = plt.subplots(1, len(self.plots), figsize=(8, 4))
        for ax, p in zip(axes, self.plots):
            p.render(ax)  # Calls child class method directly
        plt.tight_layout()
        plt.savefig("plot_framework.png")
        print("Framework plots saved to plot_framework.png")

p1 = LinePlot([1, 2, 3], [10, 20, 15], "Line Chart")
p2 = BarPlot(["A", "B", "C"], [30, 40, 25], "Bar Chart")

dash = Dashboard([p1, p2])
dash.show()
```

Q19. Student Analytics and Visualisation

Classes `Student` and `ClassAnalyzer` for calculating class averages, plotting horizontal bar charts and boxplots.

```
import matplotlib.pyplot as plt

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks  # list of 3 subjects

    def average(self):
        return sum(self.marks) / len(self.marks)

class ClassAnalyzer:
    def __init__(self, students):
        self.students = students

    def get_topper(self):
        return max(self.students, key=lambda s: s.average())

    def plot_data(self):
        # Sort students by average descending
        sorted_list = sorted(self.students, key=lambda s: s.average(), reverse=True)
        names = [s.name for s in sorted_list]
        averages = [s.average() for s in sorted_list]

        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

        # Horizontal Bar Chart
        ax1.barh(names, averages, color="teal")
        ax1.invert_yaxis()
        ax1.set_title("Student Averages")
        ax1.set_xlabel("Average Marks")

        # Subject Boxplot
        s1 = [s.marks[0] for s in self.students]
        s2 = [s.marks[1] for s in self.students]
        s3 = [s.marks[2] for s in self.students]
        ax2.boxplot([s1, s2, s3], labels=["Sub 1", "Sub 2", "Sub 3"])
        ax2.set_title("Subject Distribution")

        plt.tight_layout()
        plt.savefig("student_visuals.png")
        print("Topper:", self.get_topper().name)

# 8 Students
students_data = [
    Student("Amit", [80, 85, 90]), Student("Bina", [70, 75, 60]),
    Student("Chetan", [90, 92, 95]), Student("Divya", [60, 65, 70]),
    Student("Esha", [85, 80, 82]), Student("Fahad", [78, 82, 80]),
    Student("Geeta", [65, 70, 68]), Student("Hari", [88, 85, 90])
]
analyzer = ClassAnalyzer(students_data)
analyzer.plot_data()
```

Q20. Numerical Integration and Error Analysis

Base class `Integrator` with `Trapezoid` and `Midpoint` subclasses, evaluates across subintervals, and plots absolute error.

```
import math
import matplotlib.pyplot as plt

class Integrator:
    def __init__(self, func, a, b):
        self.func = func
        self.a = a
        self.b = b

class Trapezoid(Integrator):
    def integrate(self, n):
        h = (self.b - self.a) / n
        total = 0.5 * (self.func(self.a) + self.func(self.b))
        for i in range(1, n):
            total += self.func(self.a + i * h)
        return total * h

class Midpoint(Integrator):
    def integrate(self, n):
        h = (self.b - self.a) / n
        total = 0
        for i in range(n):
            mid = self.a + (i + 0.5) * h
            total += self.func(mid)
        return total * h

# Integrate sin(x) from 0 to pi (Exact value is 2.0)
f = math.sin
a, b = 0, math.pi
exact = 2.0

trap = Trapezoid(f, a, b)
mid = Midpoint(f, a, b)

intervals = [4, 8, 16, 32]
trap_errs = []
mid_errs = []

print("n\tTrap Error\tMid Error")
for n in intervals:
    e_trap = abs(trap.integrate(n) - exact)
    e_mid = abs(mid.integrate(n) - exact)
    trap_errs.append(e_trap)
    mid_errs.append(e_mid)
    print(f"{n}\t{round(e_trap, 5)}\t\t{round(e_mid, 5)}")

# Plotting error
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(intervals, trap_errs, marker="o", label="Trapezoid Error")
ax.plot(intervals, mid_errs, marker="s", label="Midpoint Error")
ax.set_title("Integration Error vs Subintervals")
ax.set_xlabel("Subintervals (n)")
ax.set_ylabel("Absolute Error")
ax.legend()
plt.tight_layout()
plt.savefig("integration_error.png")
print("Saved plot to integration_error.png")
```